

작업 로그 — 2026-02-26

날짜

2026-02-26

작성자

노승우

Alpamayo VRAM Optimization Research

목차 (Table of Contents)

체크리스트

[오전] 작업 이력 정리

[오전] 문서 폴더 구조 통합

[오전] md2pdf 표지 페이지 + PDF 재생성

[오전] 4-bit VRAM 제한 추가 실험 (7/9/11GB)

스왑 방식 아이디어 정리 (idea.md)

meeting-notes-01.md 전면 개편

D2H 제거 재실험 + 문서/PPTX 전면 교체

D2H 불필요 사실 반영 — 문서 전면 수정

device_map="auto" 섹션 추가 + 섹션 번호 재배정

비동기 2-Stream 파이프라인 실험 (방법 4)

스레드 기반 비동기 파이프라인 실험 (Variant D)

작업 로그 — 2026-02-26

체크리스트

- [x] 2026-02-25 작업 이력 타임스탬프 수정 및 커밋
 - [x] MD/PDF 파일 docs/ 폴더로 통합 이동
 - [x] md2pdf 표지 페이지 기능 추가 (작성자: 노승우)
 - [x] 이미지 경로 수정 + 전체 PDF 재생성
 - [x] 4-bit VRAM 제한 추가 실험 (7/9/11GB)
 - [x] 스왑 방식 아이디어 정리 (idea.md)
 - [x] 방법 1-3 검증 + 방법 4 신규성 조사
 - [x] D2H 불필요 사실 반영 — 전체 문서 수정 (idea/survey/info/overnight-report)
 - [x] meeting-notes-01.md 전면 개편 (6절 구조 + 7GB 재측정 373.05s)
 - [x] D2H 제거 재실험: 116.51s → 43.38s (6.31x 가속, 이론 예측 ~46s 근접)
 - [x] 문서/PPTX D2H 결과 전면 교체
 - [x] device_map="auto" 섹션 추가 (Section 3) + 전체 섹션 번호 재배정
 - [x] 비동기 2-Stream 파이프라인 실험 (방법 4, Variant A/C)
 - [x] 스레드 기반 비동기 파이프라인 실험 (Variant D, 방법 4 확장)
-

[오전] 작업 이력 정리

- **요청:** 어제 야간 작업 로그 타임스탬프 수정
- **상태:** 완료 ☑
- **수행 내용:**
 - 야간 실험 시간대 교정 (실험1: 23:55~01:00, 실험2: 01:00~01:31)
 - 컨텍스트 제한으로 01:31 세션 종료된 점 기록
 - 커밋: 02c7fd2

[오전] 문서 폴더 구조 통합

- 요청: 모든 MD/PDF 파일을 docs/ 폴더로 통합 관리
- 상태: 완료 ☑
- 수행 내용:
 - `work-log/*.md|pdf` → `docs/work-log/`
 - `research/*.md|pdf` + `research/0N-*/analysis.md|pdf` → `docs/research/`
 - `seminar/*.md|pdf` + `seminar/figures/` → `docs/seminar/`
 - `experiments/README.md|pdf` → `docs/experiments/`
 - `seminar/vram_breakdown.py` → `experiments/` (실험 스크립트)
- 빈 원본 폴더 정리 (`work-log/`, `seminar/` 삭제)

[오전] md2pdf 표지 페이지 + PDF 재생성

- 요청: PDF 변환 시 표지 추가, 작성자 노승우 고정
- 상태: 완료 ☑
- 수행 내용:
 - `tools/md2pdf.py` 에 표지 자동 생성 (제목/날짜/작성자/프로젝트명)
 - 이미지 경로 수정 (04-creative-approaches figures 이동)
 - docs/ 전체 18개 MD → PDF 재생성
- 커밋: 7d89fa7

[오전] 4-bit VRAM 제한 추가 실험 (7/9/11GB)

- 요청: 기존 4개 조건(12/10/8/6GB)에 7/9/11GB 추가
- 상태: 완료 ☑
- 수행 내용:
 - `run_vram_limit_test_additional.py` 작성 및 실행
 - 기존 결과와 병합 (7개 조건, VRAM 내림차순)
 - 시각화 3종 재생성
- 결과:
 - 11GB: 5.29s (1.1x) — baseline과 유사
 - 9GB: 111.36s (23.2x) — **Cliff 시작점 확인** (10GB→9GB 구간)

- 7GB: 4.76s (1.0x) — 이상치 (Unified Memory 전면 사용으로 swap 오버헤드 감소 추정)
- Performance Cliff는 모델 크기(8.87GB) 근처인 9~10GB 구간에서 발생

스왑 방식 아이디어 정리 (idea.md)

- 요청: 스왑 방식 세 가지 + 추가 제안을 문서화
- 상태: 완료
- 수행 내용:
 - docs/research/idea.md 작성 — 방법 1~4 정리
 - 방법 1(레이어 단위): 기존 구현과 정확히 일치
 - 방법 2(최적 청크): 벤치마크 결과와 정확히 일치
 - 방법 3(모듈 단위): VLM 15.17GB > 12GB VRAM으로 실험적 불가 판정
 - 방법 4(비동기 파이프라인): 선행 연구 조사 완료

meeting-notes-01.md 전면 개편

- 요청: 교수님 미팅 자료를 체계적 연구 보고서 형태로 재구성
- 상태: 완료 ☑
- 수행 내용:
 - 기존 단순 정리 → 6절 구조(VRAM 분석/스왑 오버헤드/Demand Layering/방법론 비교/종합/WSL2)
 - 4-bit VRAM 제한 실험 데이터 + Demand Layering 결과 통합
 - 4가지 스왑 방법론 이론적 실행시간 비교 테이블 추가
 - 교수님 2-Stage 아이디어 분석 포함
 - **7GB 재측정 완료**: 기존 4.76s(이상치) → **373.05s** (정상 확인)
 - 시각화 4종 갱신 (vram_limit_comparison, performance_cliff, transfer_breakdown, demand_layering_comparison)
 - 검증 에이전트로 전체 수치 교차 검증 완료
 - PDF 재생성 (844KB)

D2H 제거 재실험 + 문서/PPTX 전면 교체

- 요청: D2H를 실제로 제거한 구현으로 재실험 + 결과를 문서에 반영

- 상태: 완료 ☑
- 실험 결과:
 - `research/07-demand-layering-no-d2h/demand_layering_no_d2h.py` 작성
 - `module.to("cpu")` → CPU 원본 참조 복원 + GPU free 방식
 - **43.38s** (기존 D2H 포함 116.51s → **2.69x 개선**)
 - H2D: 450회, 총 24.23s (avg 53.83ms) — 기존 28.9s에서 개선
 - Free: 450회, 총 1.05s (avg 2.32ms) — 기존 D2H 70.5s → 제거
 - 이론 예측(~46s)보다 빠른 43.38s 달성
- 문서 수정:
 - meeting-notes-01.md: 3절 실험 결과, 4절 방법론 비교, 5절 종합 테이블 전면 교체
 - PPTX: 동일 수치 반영 (27슬라이드)
 - PDF 재생성

D2H 불필요 사실 반영 — 문서 전면 수정

- 요청: 파라미터 오프로딩 추론에서 D2H copy 불필요 사실을 전체 문서에 반영
- 상태: 완료
- 핵심 통찰: CPU에 파라미터 원본이 유지되므로 GPU에서 free만 하면 됨
- 현재 구현의 D2H 70.5s는 완전히 제거 가능한 오버헤드
- 3-stream → 2-stream(compute + H2D)으로 단순화
- 수정 파일:
 - `docs/research/idea.md` — 방법 4를 2-Stream으로 전면 수정
 - `docs/research/async-usage-tracking-swap-survey.md` — 제안 요약, 비교 테이블, 섹션 4-6 수정
 - `docs/info.md` — CUDA stream 설명, 레이어 오프로딩 테이블 수정
 - `docs/research/overnight-work-report.md` — D2H 불필요 주석 추가
- 전체 4개 파일 PDF 재생성

device_map="auto" 섹션 추가 + 섹션 번호 재배정

- 요청: device_map="auto" 관련 내용 보충
- 상태: 완료 ☑
- 수행 내용:

- meeting-notes-01.md에 **Section 3: 기존 오프로딩 프레임워크의 한계** 신설
- 내용: device_map="auto" 동작, 추론 실패 원인(fuse_traj_tokens/masked_scatter), 수동 오프로딩 실패, 커스텀 구현 필요성
- 기존 Section 3~6 → Section 4~7로 전체 번호 재배정
- PPTX: device_map 슬라이드 2장 추가 + 목차/섹션 번호 업데이트
- PDF 재생성

비동기 2-Stream 파이프라인 실험 (방법 4)

- **요청:** 비동기 방식 실험
- **상태:** 완료 ☑
- **실험 스크립트:** `research/08-async-pipeline/async_pipeline.py`
- **결과:**
 - Variant A (Pageable async): 44.16s — 개선 없음 (pageable에서 non_blocking 무효)
 - Variant C (Staged pinned buffer): 38.45s (1.13x vs 43.38s baseline)
 - DMA 오버랩 성공 (wait_dma 0.05ms), CPU staging(44ms)이 새 병목
 - 16GB RAM 한도로 전체 파라미터 pinning 불가

스레드 기반 비동기 파이프라인 실험 (Variant D)

- **요청:** Variant C의 CPU memcpy 병목을 background thread로 오버랩하여 추가 가속
- **상태:** 완료 ☑
- **방법:** Variant C의 full-layer pinned double buffer 방식을 background thread에서 실행
- CPU memcpy(~44ms)를 background thread에서 수행 → GPU compute와 오버랩
- Python GIL은 C++ `tensor.copy_()` 실행 중 해제되므로 진정한 병렬 실행 가능
- **결과:**
 - **per-pass 2.41s** (Baseline 2.89s, Variant C 2.96s) → **약 1.20x per-pass 개선**
 - Peak VRAM: 11.08 GB (안전)
 - CPU memcpy의 GPU compute 오버랩 성공
 - DMA latency(~46ms)는 memcpy 완료 후에야 시작되어 오버랩되지 않음
 - **Chunked Transfer 시도 — 실패:**
 - chunk-level pipelining (4MB 청크, ring buffer): 파라미터 corruption 발생
 - per-param interleaving (param별 memcpy→DMA 교차): 파라미터 corruption 발생

- 원인: background thread에서 `with torch.cuda.stream()` 내 CPU memcpy와 CUDA DMA 혼합 시 데이터 정합성 문제
- chunking 없이 순차 실행(all memcpy → all DMA)만 정상 동작
- **종합 비교 (per-pass normalized):**

방법	per-pass	vs Baseline
Baseline (Method 1, sync)	2.89s	1.00x
Variant C (staged pinned)	2.96s	0.98x
Variant D (threaded)	2.41s	1.20x

- **문서 업데이트:** meeting-notes-01.md Section 5.5 / Section 6 반영 완료